System Design Experiment 2

High-Level Design of Netflix

1 Functional Requirements

1. **User Management**
   - Users can sign up, log in, and log out.
   - Users can create and manage multiple profiles under one account.
   - Users can update profile settings and preferences.
2. **Content Discovery**
   - Users can browse movies, TV shows, and other available content.
   - Users can search for content by title, genre, actor, or category.
   - The system should provide personalized content recommendations.
   - Users can view content details such as title, description, cast, genre, rating, and duration.
3. **Video Streaming**
   - Users can play movies and TV shows on supported devices.
   - Users can pause, resume, seek, and stop playback.
   - The system should support adaptive video quality based on network conditions.
   - Users should be able to resume content from where they previously stopped.
4. **Content Management**
   - Administrators or content providers can add, update, and remove content.
   - The system should store metadata, thumbnails, subtitles, and video files.
   - The system should support multiple video qualities and formats.
5. **Watchlist and History**
   - Users can add or remove content from their watchlist.
   - The system should maintain users' viewing history.
   - Users can continue watching previously started content.
6. **Ratings and Feedback**

- Users can rate or provide feedback on content.
- The system should use viewing activity and feedback to improve recommendations.

7. **Subscription and Payments**
   - Users can select and manage subscription plans.
   - The system should process subscription payments.
   - The system should handle subscription renewal, cancellation, and expiration.
8. **Notifications**
   - The system can notify users about new releases, recommendations, and subscription-related events.

2 Non-Functional Requirements

1. **Scalability**
   - The system should support millions of concurrent users and streaming sessions.
   - Services should be horizontally scalable to handle traffic growth.
2. **High Availability**
   - The system should remain available even if individual services or servers fail.
   - There should be minimal service interruption during failures.
3. **Reliability**
   - Video playback should be reliable with minimal buffering and failures.
   - The system should tolerate failures in individual components.
4. **Performance**
   - Content browsing and search should have low response latency.
   - Video playback should start quickly after the user requests a video.
   - Recommendations should be generated with low latency.
5. **Fault Tolerance**
   - Failure of a server, service, or storage node should not bring down the entire system.
   - The system should support automatic recovery and failover.
6. **Security**
   - User credentials and personal information must be protected.
   - Communication between clients and servers should be encrypted.
   - Payment information must be handled securely.

- Only authorized users and services should be able to access protected resources.

7. **Data Consistency**
   - Critical data such as subscriptions, payments, and user accounts should maintain strong consistency.
   - Less critical data such as recommendations can tolerate eventual consistency.
8. **Durability**
   - User data, content metadata, and viewing history should not be lost in case of hardware or service failures.
   - Data should be replicated and backed up.
9. **Maintainability**
   - The system should be modular and use loosely coupled services.
   - Individual services should be deployable and updated independently.
10. **Global Availability**
    - The platform should be accessible to users across different geographical regions.
    - Content should be served from locations close to users to reduce latency.
11. **Observability**
    - The system should provide logging, monitoring, metrics, and distributed tracing.
    - Failures and performance issues should be detected quickly.

3 Back-of-the-Envelope Estimation

3.1 Assumptions

- Total registered users: 300 million
- Daily active users (DAU): 100 million
- Average streaming time per active user: 2 hours/day
- Peak traffic is approximately 3 times the average traffic.
- Average video bitrate: 5 Mbps
- Average API request rate per active user: 10 requests/hour

3.2 Daily Streaming Hours

$$100 \text{ million users} \times 2 \text{ hours/day} = 200 \text{ million streaming hours/day}$$

3.3 Average Concurrent Viewers

$$\frac{200 \text{ million hours/day} \times 60 \times 60}{86400 \text{ seconds/day}} \approx 2.3 \text{ million concurrent viewers}$$

Therefore, the system should support approximately 2.3 million concurrent viewers on average.

3.4 Peak Concurrent Viewers

$$2.3 \text{ million} \times 3 \approx 7 \text{ million concurrent viewers}$$

Therefore, the system should be designed to support approximately 7 million concurrent streaming sessions during peak traffic.

3.5 Video Bandwidth

Assuming an average video bitrate of 5 Mbps:

$$7 \text{ million} \times 5 \text{ Mbps} = 35,000,000 \text{ Mbps} = 35 \text{ Tbps}$$

Therefore, the system may need to deliver approximately 35 Tbps of peak video traffic. This traffic should primarily be handled by a CDN rather than application servers.

3.6 API Requests

Assuming each active user generates approximately 10 API requests per hour:

$$100 \text{ million} \times 10 = 1 \text{ billion requests/day}$$

Average requests per second:

$$\frac{1 \text{ billion}}{86400} \approx 11,600 \text{ requests/second}$$

Peak requests per second:

$$11,600 \times 3 \approx 35,000 \text{ requests/second}$$

Therefore, the backend should be designed to handle approximately 35K API requests/second during peak traffic.

3.7 Storage for Video Content

Assume:

- 20,000 movies and shows
- Average encoded video size = 10 GB

$$20,000 \times 10 \text{ GB} = 200,000 \text{ GB} = 200 \text{ TB}$$

With multiple video qualities, replicas, subtitles, thumbnails, and additional content, actual storage requirements can reach several petabytes.

Therefore, object storage such as Amazon S3 can be used for large-scale video storage.

3.8 User Data Storage

Assume:

- 300 million registered users
- Average user/profile data = 10 KB

$$300 \text{ million} \times 10 \text{ KB} \approx 3 \text{ TB}$$

Additional storage will be required for viewing history, watchlists, preferences, ratings, and other metadata.

4 Components Used

1. **Client / Mobile / Web Application**
   - Provides the user interface for browsing, searching, and streaming content.
   - Sends API requests to the backend services.
2. **DNS**
   - Resolves the Netflix domain to the appropriate servers or services.
   - Helps route users to the nearest or appropriate region.
3. **CDN (Content Delivery Network)**
   - Delivers video content from edge locations close to users.
   - Reduces latency and buffering.
   - Handles the majority of video streaming traffic.
4. **Load Balancer**
   - Distributes incoming API traffic across multiple backend servers.
   - Prevents individual servers from becoming overloaded.
   - Provides high availability and failover.
5. **API Gateway**
   - Acts as the entry point for client API requests.
   - Handles routing, authentication, rate limiting, and request management.
6. **Authentication / User Service**
   - Handles user login, signup, authentication, and profile management.
   - Stores user and profile information.
7. **Content Service**
   - Manages movies, TV shows, metadata, genres, thumbnails, subtitles, etc.

- Provides content information to users.

8. **Search Service**
   - Handles content search.
   - Uses an indexing system to provide fast search results.
9. **Recommendation Service**
   - Generates personalized movie and TV show recommendations.
   - Uses user behavior, viewing history, ratings, and content metadata.
10. **Streaming / Playback Service**
    - Manages video playback-related operations.
    - Handles playback authorization and related control-plane operations.
11. **Object Storage**
    - Stores large video files, different video resolutions, thumbnails, and subtitles.
    - Example: Amazon S3.
12. **Database**
    - Stores structured data such as users, profiles, subscriptions, content metadata, watchlists, and viewing history.
    - A combination of SQL and NoSQL databases can be used depending on the data.
13. **Cache**
    - Stores frequently accessed data such as popular content, metadata, and sessions.
    - Reduces database load and improves response time.
    - Example: Redis.
14. **Message Queue / Event Streaming**
    - Handles asynchronous events such as viewing activity, recommendations, notifications, and analytics.
    - Decouples services and improves scalability.
    - Example: Apache Kafka.
15. **Payment / Subscription Service**
    - Manages subscription plans, payments, renewals, cancellations, and expiration.
16. **Notification Service**
    - Sends emails, push notifications, and other user notifications.

17. **Analytics / Data Processing System**
    - Collects viewing and user interaction events.
    - Processes large-scale data for analytics and recommendation systems.
18. **Monitoring and Logging**
    - Collects metrics, logs, and traces.
    - Helps detect failures, performance issues, and service outages.

5 Flow and Connections

5.1 User Access Flow

1. User opens the Netflix application.
2. The client sends a request to DNS.
3. DNS resolves the request and directs the user to the appropriate region or service.
4. The request reaches the Load Balancer/API Gateway.
5. The API Gateway authenticates the request and routes it to the required backend service.

5.2 Login / Authentication Flow

User → API Gateway → Authentication/User Service → Database

- User submits login credentials.
- Authentication service verifies the credentials.
- A user session or token is created.
- Authentication information is returned to the client.

5.3 Content Browsing Flow

User → API Gateway → Content Service → Cache/Database

- User requests the home page or content information.
- Content Service checks the cache for frequently accessed information.
- If data is not available in the cache, it is retrieved from the database.
- The response is returned to the user.

5.4 Search Flow

User → API Gateway → Search Service → Search Index

- User enters a search query.
- API Gateway forwards the request to the Search Service.
- Search Service queries the search index.
- Matching movies and shows are returned to the client.

5.5 Recommendation Flow

User → API Gateway → Recommendation Service

- User requests personalized recommendations.
- Recommendation Service uses user preferences, viewing history, ratings, and other behavioral data.
- Recommendations are returned to the client.

5.6 Video Streaming Flow

User → API Gateway → Streaming/Playback Service

Streaming/Playback Service → CDN → Object Storage

- User selects a movie or TV show.
- The client requests playback authorization and metadata from the backend.
- Streaming Service verifies that the user has access to the content.
- The client receives the required manifest and playback information.
- The client requests video segments from the CDN.
- CDN serves video segments from the nearest edge location.
- If the required content is not present at the CDN edge, it is fetched from origin/object storage and cached at the CDN.
- Video is delivered directly from the CDN to the user.

5.7 Adaptive Streaming Flow

User ↔ CDN

- Video is stored in multiple resolutions and bitrates.
- The client monitors network speed and device capabilities.
- The client requests an appropriate video segment quality.
- Quality can automatically increase or decrease depending on network conditions.

5.8 Viewing Activity Flow

$$\text{User} \rightarrow \text{API Gateway} \rightarrow \text{Playback/Activity Service} \rightarrow \text{Message Queue}$$

$$\text{Message Queue} \rightarrow \text{Data Processing} \rightarrow \begin{cases} \text{Analytics} \\ \text{Recommendation System} \end{cases}$$

- Playback events such as play, pause, completion, and watch duration are generated.
- Events are sent asynchronously through a message queue or event streaming system.
- Data processing systems consume these events.
- Processed data is used for analytics and improving recommendations.

5.9 Watchlist / Viewing History Flow

$$\text{User} \rightarrow \text{API Gateway} \rightarrow \text{User/History Service} \rightarrow \text{Cache/Database}$$

- User adds or removes content from the watchlist.
- Viewing progress and history are updated.
- Frequently accessed information can be cached.
- Persistent data is stored in the database.

5.10 Subscription and Payment Flow

$$\text{User} \rightarrow \text{API Gateway} \rightarrow \text{Subscription Service} \rightarrow \text{Payment Service} \rightarrow \text{Database}$$

- User selects a subscription plan.
- Subscription Service sends the payment request to the Payment Service.
- Payment is processed through an external payment provider.
- Subscription status is updated after successful payment.
- The user's access rights are updated accordingly.

5.11 Failure Handling

- Load Balancers distribute traffic across multiple instances.
- Services are replicated across multiple servers and availability zones.
- CDN provides geographically distributed content delivery.
- Databases use replication and backups for durability.
- Failed service instances can be replaced automatically.

6 Conclusion

The proposed Netflix High-Level Design uses a distributed and scalable architecture to support millions of concurrent users. The architecture separates the control/API path from the video delivery path. Backend services handle authentication, content discovery, search, recommendations, subscriptions, and playback authorization, while the CDN handles large-scale video delivery.

The use of caching, load balancing, message queues, replicated databases, object storage, and CDNs helps achieve scalability, high availability, fault tolerance, low latency, and reliable video streaming.